ĐẠI HỌC QUỐC GIA HÀ NỘI

ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO THỰC HÀNH 3

Sinh viên thực hiện: Trịnh Quang Sáng

Mã sinh viên: 23020867

Giảng viên hướng dẫn: TS Hoàng Gia Hưng

KS Đỗ Đình Minh

Hà Nội – 2026

1 Bài 1: Quan sát không gian địa chỉ của tiến trình

1.1 Mã nguồn

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_value = 100;
int global_variable;

int main() {
    static int static_local_value = 10;
    int local_value = 15;

    int *heap_value = malloc(sizeof(int));
    if (heap_value == NULL) {
        perror("malloc");
        return 1;
    }
    *heap_value = 20;

    printf("Process ID: %d\\n", getpid());
    printf("Global value: %p\\n", (void *)&global_value);
    printf("Global variable: %p\\n", (void *)&global_variable);
    printf("Static local value: %p\\n", (void *)&static_local_value);
    printf("Local value: %p\\n", (void *)&local_value);
    printf("Heap value: %p\\n", (void *)heap_value);

    getchar();
    free(heap_value);
    return 0;
}
```

1.2 Kết quả và nhận xét

Biên dịch chương trình bằng lệnh `gcc -Wall -Wextra -o memory_layout memory_layout.c`, sau đó chạy bằng `./memory_layout`. Kết quả ghi nhận trong lần chạy được minh họa ở Hình 1.

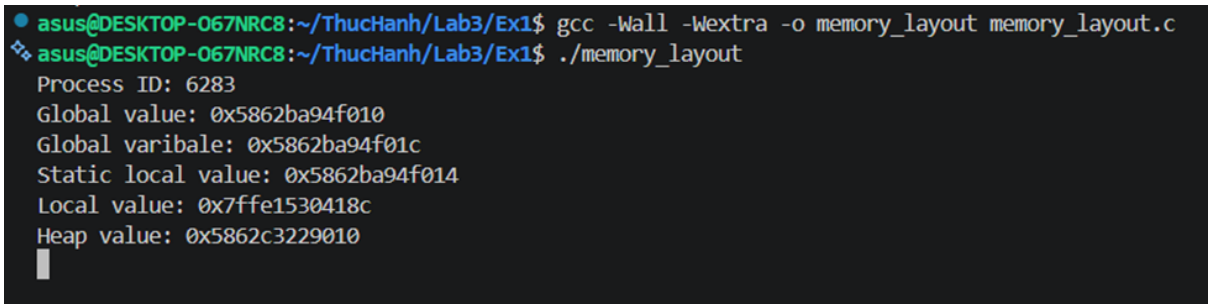


Figure 1: Kết quả in PID và địa chỉ các biến khi chương trình chạy

Kết quả in ra process id của tiến trình đang chạy và địa chỉ ảo của các biến được khởi tạo.

- Biến `global_value` được khởi tạo tường minh nên được lưu trong vùng nhớ data segment.
- Biến `global_variable` không được gán giá trị khi khởi tạo nên được coi là khởi tạo không tường minh, giá trị được gán ngầm là 0 nên giá trị được lưu trong BSS.

- Biến `static_local_value` có phạm vi sử dụng trong hàm main nhưng lại là biến static, tồn tại trong suốt vòng đời của tiến trình nên được lưu trong data segment.
- Biến `local_value` là biến cục bộ và không phải là static, nên địa chỉ của biến ở vùng stack segment.
- Biến `heap_value` được cấp phát động bằng malloc, nên được lưu trong vùng nhớ heap của bộ nhớ.

Tiến trình đang chạy có pid là 6283, sử dụng lệnh `cat /proc/6283/maps` để xem sơ đồ phân bổ bộ nhớ ảo của tiến trình.

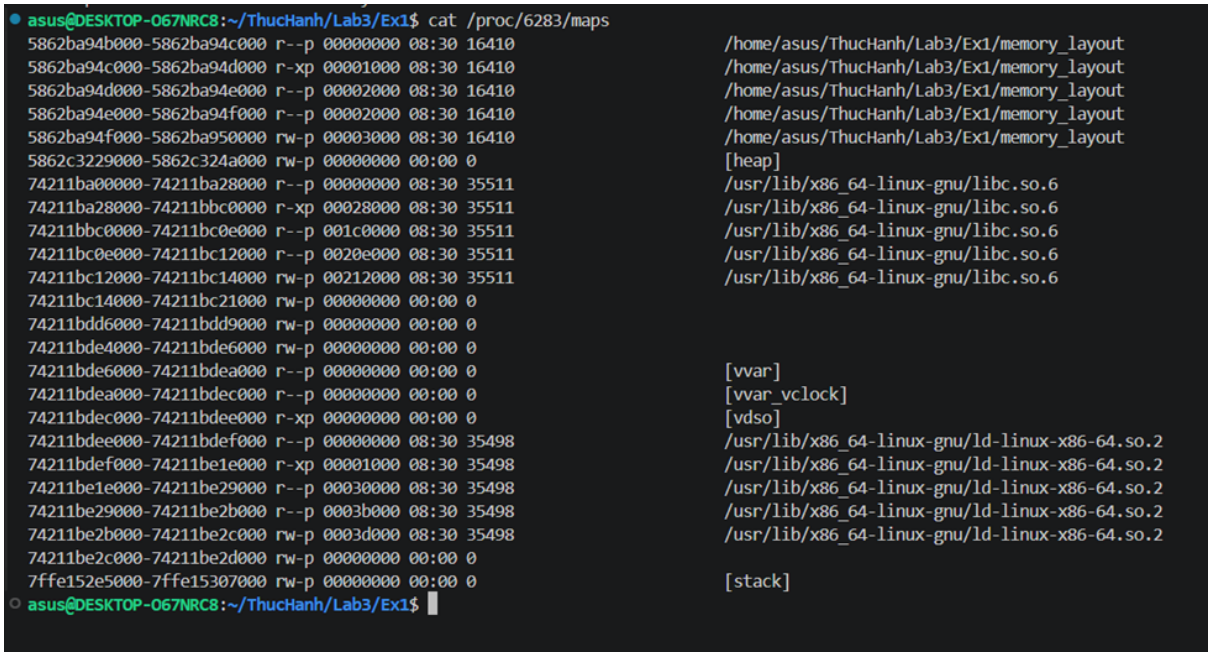


Figure 2: Sơ đồ phân bổ bộ nhớ ảo của tiến trình PID 6283

Các địa chỉ ảo mà chương trình in ra đều nằm trong dải địa chỉ bộ nhớ bên trên. Ngoài các vùng stack và heap được đánh dấu rõ ràng thì các vùng data và BSS không có nhãn riêng lẻ.

1.3 Phân tích kết quả

Bảng 1 tổng hợp địa chỉ quan sát được khi chạy chương trình và vùng nhớ tương ứng dựa trên sơ đồ bộ nhớ của tiến trình.

Đối tượng	**Địa chỉ quan sát được**	**Vùng nhớ dự kiến**
Biến toàn cục đã khởi tạo (`global_value`)	`0x5862ba94f010`	Data segment (thuộc vùng `5862ba94f000-5862ba950000`)
Biến toàn cục chưa khởi tạo (`global_variable`)	`0x5862ba94f01c`	BSS segment (thuộc vùng `5862ba94f000-5862ba950000`)
Biến static (`static_local_value`)	`0x5862ba94f014`	Data segment (thuộc vùng `5862ba94f000-5862ba950000`)
Biến cục bộ (`local_value`)	`0x7ffe1530418c`	Stack (thuộc vùng `7ffe152e5000-7ffe15307000`)
Vùng nhớ cấp phát bằng `malloc()` (`heap_value`)	`0x5862c3229010`	Heap (thuộc vùng `5862c3229000-5862c324a000`)

Table 1: Địa chỉ quan sát được và vùng nhớ dự kiến của các biến

Nhận xét: Cấu trúc phân bố bộ nhớ của tiến trình có PID 6283 hoàn toàn giống với phân bố lý thuyết. Các biến toàn cục và static được phân bổ ở địa chỉ thấp, trong vùng Data hoặc BSS tùy thuộc vào việc chúng có được khởi tạo tường minh hay không. Cao hơn là vùng chia sẻ giữa stack và heap. Heap sẽ mở rộng từ đầu địa chỉ thấp về phía địa chỉ cao cho đến khi gặp vùng nhớ stack, còn stack thì ngược lại, mở rộng từ vùng địa chỉ cao xuống vùng địa chỉ thấp hơn cho đến khi gặp heap.

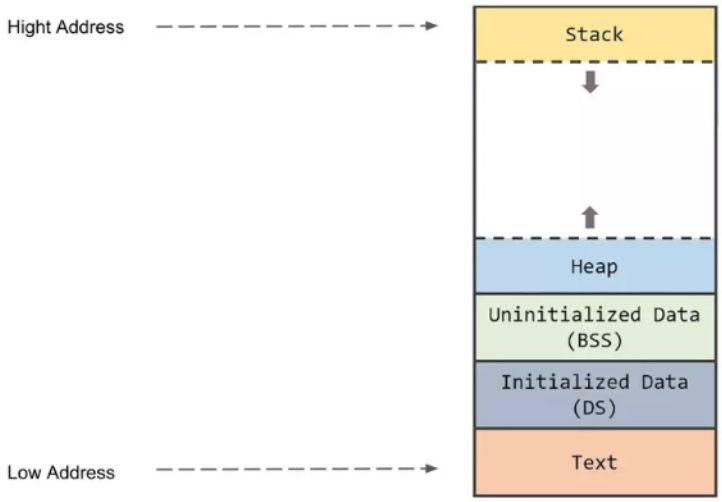


Figure 3: Sơ đồ lý thuyết về cấu trúc phân bố bộ nhớ của tiến trình

1.4 Kiểm tra sự thay đổi địa chỉ

Chương trình được chạy ba lần để so sánh PID và địa chỉ của các biến. Kết quả cho thấy PID cùng các địa chỉ ảo thay đổi giữa các lần chạy, trong khi các biến toàn cục và static vẫn giữ nguyên độ lệch tương đối trong cùng một vùng nhớ. Sự thay đổi này phù hợp với cơ chế ngẫu nhiên hóa không gian địa chỉ (ASLR).

```
asus@DESKTOP-O67NRC8:~/ThucHanh/Lab3/Ex1$ gcc -Wall -Wextra -o memory_layout memory_layout.c
asus@DESKTOP-O67NRC8:~/ThucHanh/Lab3/Ex1$ ./memory_layout
Process ID: 16944
Global value: 0x5d102cc2f010
Global varibale: 0x5d102cc2f01c
Static local value: 0x5d102cc2f014
Local value: 0x7ffd2640fb4c
Heap value: 0x5d10421b8010

asus@DESKTOP-O67NRC8:~/ThucHanh/Lab3/Ex1$
```

Figure 4: Kết quả kiểm tra địa chỉ trong lần chạy thứ nhất

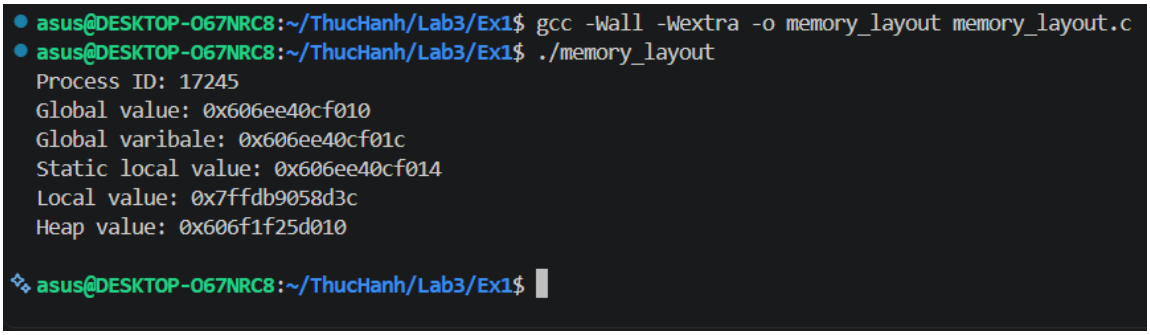


Figure 5: Kết quả kiểm tra địa chỉ trong lần chạy thứ hai

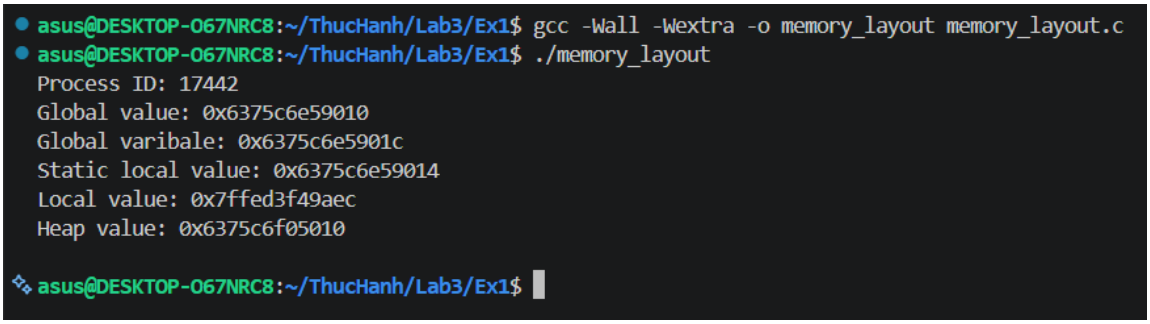


Figure 6: Kết quả kiểm tra địa chỉ trong lần chạy thứ ba

2 Bài 2: Theo dõi lời gọi hệ thống bằng `strace`

2.1 Mã nguồn

Mã nguồn của chương trình trong tệp `file_trace.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(){
    //Mở file với các cờ đọc và ghi, tạo file nếu chưa tồn tại,
    //xóa nội dung cũ nếu file đã tồn tại
    int fd = open("data.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);

    //Kiểm tra giá trị hợp lệ của fd
    if(fd == -1){
        perror("File descriptor");
        return 1;
    }

    char message[13] = "Linux Driver";
    char buffer[13] = {0};

    //Ghi nội dung message vào file data.txt và kiểm tra giá trị trả về
    if(write(fd, message, sizeof(message) - 1) != sizeof(message) - 1){
        perror("write");
        close(fd);
        return 1;
    }

    //Dịch con trỏ file về vị trí đầu file và kiểm tra giá trị trả về
    if(lseek(fd, 0, SEEK_SET) == -1){
        perror("lseek");
        close(fd);
        return 1;
    }

    //Đọc giá trị từ file data.txt vào mảng buffer và kiểm tra giá trị trả về
    int cnt = read(fd, buffer,sizeof(buffer) - 1);
    if(cnt == -1){
        perror("read");
        close(fd);
        return 1;
    }

    //In ra dữ liệu đọc được và số byte đọc thực tế
    printf("Buffer read: %s\n", buffer);
    printf("Byte: %d\n", cnt);

    //Đóng files
```

```
    close(fd);
}
```

2.2 Kết quả chạy và theo dõi lời gọi hệ thống

Chương trình in ra nội dung `Linux Driver` và số byte đọc được là 12. Dùng lệnh `strace -e trace=openat,read,write,lseek,close ./file_trace` để theo dõi các lời gọi hệ thống. Log cho thấy chương trình mở `data.txt` bằng `openat()`, ghi 12 byte bằng `write()`, đưa vị trí đọc về đầu tệp bằng `lseek()`, đọc lại 12 byte bằng `read()`, in kết quả ra terminal và đóng tệp bằng `close()`. Các lời gọi `openat()` và `read()` xuất hiện trước đó là một phần của quá trình nạp thư viện động.

```
asus@DESKTOP-O67NRC8:~/ThucHanh/Lab3/Ex2$ ./file_trace
Buffer read: Linux Driver
Byte: 12
asus@DESKTOP-O67NRC8:~/ThucHanh/Lab3/Ex2$ strace -e trace=openat,read,write,lseek,close ./file_trace
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "data.txt", O_RDWR|O_CREAT|O_TRUNC, 0644) = 3
write(3, "Linux Driver", 12)            = 12
lseek(3, 0, SEEK_SET)                   = 0
read(3, "Linux Driver", 12)             = 12
write(1, "Buffer read: Linux Driver\n", 26Buffer read: Linux Driver
) = 26
write(1, "Byte: 12\n", 9Byte: 12
)               = 9
close(3)                                = 0
+++ exited with 0 +++
asus@DESKTOP-O67NRC8:~/ThucHanh/Lab3/Ex2$
```

Figure 7: Kết quả chạy chương trình và theo dõi lời gọi hệ thống bằng `strace`

2.3 Phân tích kết quả

Bảng 2 tổng hợp các lời gọi hệ thống chính của chương trình, các tham số quan trọng, giá trị trả về và chức năng tương ứng theo kết quả `strace`.

System call	Tham số quan trọng	Giá trị trả về	Ý nghĩa
`openat()`	`AT_FDCWD`, `data.txt`, `O_RDWR | O_CREAT | O_TRUNC`, `0644`	`3`	Mở hoặc tạo tệp `data.txt` trong thư mục hiện tại với quyền đọc/ghi; xóa nội dung cũ nếu tệp đã tồn tại. Trả về file descriptor là `3`.
`write()`	`3`, `Linux Driver`, `12`	`12`	Ghi 12 byte từ chuỗi `Linux Driver` vào tệp có fd = `3`. Trả về số byte đã ghi thành công.
`lseek()`	`3`, `0`, `SEEK_SET`	`0`	Đưa vị trí hiện tại của tệp có fd là `3` về đầu tệp để chuẩn bị đọc. Trả về vị trí mới là `0`.
`read()`	`3`, `buffer`, `12`	`12`	Đọc tối đa 12 byte từ tệp có fd = `3` vào `buffer`. Trả về số byte đọc thành công là `12`.
`close()`	`3`	`0`	Đóng tệp có fd `3` và giải phóng tài nguyên hệ thống. Trả về `0` khi đóng thành công.

Table 2: Phân tích các lời gọi hệ thống chính trong chương trình

Dựa vào kết quả:

- File descriptor được cấp khi mở `data.txt` là 3.
- Số byte thực tế được ghi và đọc đều là 12 byte.
- Giá trị offset sau thao tác `lseek()` là 0, chính là vị trí của con trỏ tệp mới sau khi được đặt lại tính từ đầu tệp.
- Ý nghĩa của giá trị trả về 0 từ `close()`: giá trị 0 biểu thị đã đóng tệp thành công, không gặp lỗi.

2.4 Phân tích đường đi qua VFS

Khi chương trình gọi `read(fd, buffer, 12)`, lời gọi đi từ user space vào kernel space qua bộ xử lý lời gọi hệ thống. VFS dùng file descriptor để tra cứu đối tượng `struct file` tương ứng, sau đó chuyển thao tác đọc đến phương thức phù hợp của hệ thống tệp hoặc thiết bị. Dữ liệu được đọc và chuyển về vùng nhớ của tiến trình để lưu vào `buffer`; sơ đồ minh họa bước chuyển dữ liệu này bằng `copy_to_user()`. Chi tiết xử lý phía dưới VFS phụ thuộc vào loại tệp và hệ thống tệp.

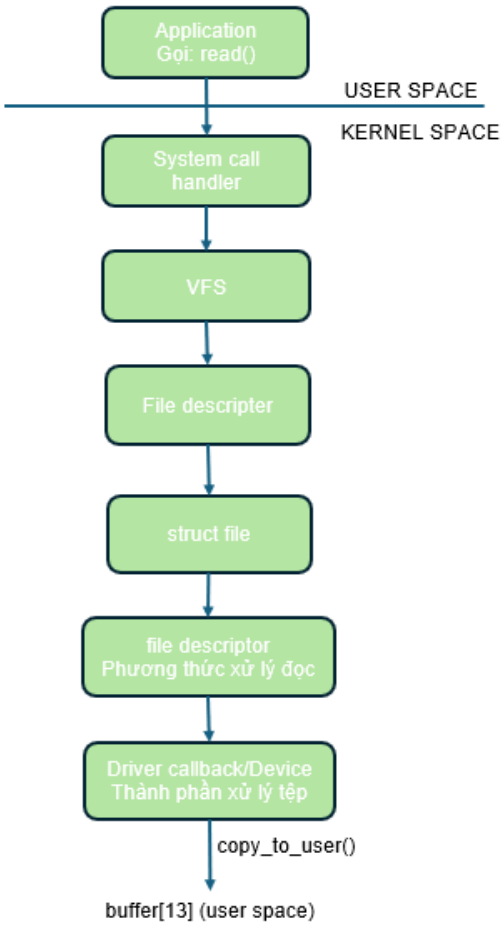


Figure 8: Luồng xử lý tổng quát của lời gọi `read()` qua VFS

- **Application và System call handler:** từ user space, thực hiện lời gọi hàm `read()` và truyền vào các tham số file descriptor, địa chỉ vùng đệm lưu dữ liệu và số byte cần đọc. Sau đó, System call handler sẽ tiếp nhận và đưa các tham số này xuống Kernel space.
- **VFS và file descriptor:** VFS tra cứu file descriptor được truyền vào để tìm `struct file` dựa trên file descriptor table.
- **Struct file:** từ file desciptor, kernel tìm được đối tượng `struct file`, có chứa cấu trúc thông tin trạng thái mở tệp.
- VFS thông qua con trỏ trong `struct file` để tìm đến cấu trúc `file_operations` để xác định phương thức xử lý đọc tương ứng với đối tượng tệp.
- **Thành phần xử lý tệp:** Yêu cầu được chuyển đến driver. Driver lấy dữ liệu và sử dụng hàm `copy_to_user()` để sao chép ngược dữ liệu lên vùng nhớ user space. Sau đó, hàm trả về số byte đọc được thành công.

3 Bài 3: Tìm hiểu file descriptor và file offset

3.1 Mã nguồn

Mã nguồn của chương trình trong tệp `file_offset.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(){

    //Yêu cầu 2: mở cùng 1 tệp 2 lần
    int fd1 = open("sample.txt", O_RDONLY);
    int fd2 = open("sample.txt", O_RDONLY);
    int fd3 = dup(fd1);
    if(fd1 == -1 || fd2 == -1){
        perror("open");
        close(fd1);
        close(fd2);
        return 1;
    }

    printf("Yeu cau 2: mo cung 1 tep 2 lan\n");
    char buf[20] = {0};
    //Đọc 2 ký tự của fd1
    if(read(fd1, buf, 2) == -1){
        perror("read fd1");
        close(fd1);
        close(fd2);
        return 1;
    }
    //In ra dữ liệu đọc được và offset file của fd1
    printf("File descriptor 1: %s\n", buf);
    printf("Offset fd1: %ld\n", lseek(fd1, 0, SEEK_CUR));

    //Đọc 2 ký tự của fd2
    if(read(fd2, buf, 2) == -1){
        perror("read fd2");
        close(fd1);
        close(fd2);
        return 1;
    }
    //In ra dữ liệu đọc đc và offset file của fd2
    printf("File descriptor 2: %s\n", buf);
    printf("Offset fd2: %ld\n", lseek(fd2, 0, SEEK_CUR));
    printf("-------------------------------\n");

    //Yêu cầu 3: tạo thêm 1 descriptor bằng dup
    printf("Yeu cau 3: tao them 1 fd bang dup\n");
    //int fd3 = dup(fd1);
```

```
//Đọc 2 ký tự của fd3
if(read(fd3, buf, 2) == -1){
    perror("read f3");
    close(fd1);
    close(fd2);
    close(fd3);
}

//In ra dữ liệu đọc đc và offset file của fd3
printf("File descriptor 3: %s\n", buf);
printf("Offset fd3: %ld\n", lseek(fd3, 0, SEEK_CUR));

 if(read(fd1, buf, 2) == -1){
    perror("read fd1");
    close(fd1);
    close(fd2);
    return 1;
}
//In ra dữ liệu đọc được và offset file của fd1
printf("File descriptor 1: %s\n", buf);
printf("Offset fd1: %ld\n", lseek(fd1, 0, SEEK_CUR));

//Đọc 2 ký tự của fd2
if(read(fd2, buf, 2) == -1){
    perror("read fd2");
    close(fd1);
    close(fd2);
    return 1;
}
//In ra dữ liệu đọc đc và offset file của fd2
printf("File descriptor 2: %s\n", buf);
printf("Offset fd2: %ld\n", lseek(fd2, 0, SEEK_CUR));
printf("--------------------------------\n");

//Yêu cầu 4: Đưa file offset của fd1 về 0

printf("Yeu cau 4\n");
if(lseek(fd1, 0, SEEK_SET) < 0){
    perror("lseek");
    close(fd1);
    close(fd2);
    close(fd3);
    return 1;
}

//In ra offset của 3 fd
printf("Offset fd2: %ld\n", lseek(fd2, 0, SEEK_CUR));
printf("Offset fd3: %ld\n", lseek(fd3, 0, SEEK_CUR));
printf("--------------------------------\n");
```

```
    close(fd1);
    close(fd2);
    close(fd3);
}
```

3.2 Kết quả chạy

Kết quả biên dịch và chạy chương trình `file_offset.c` được minh họa trong Hình 9.

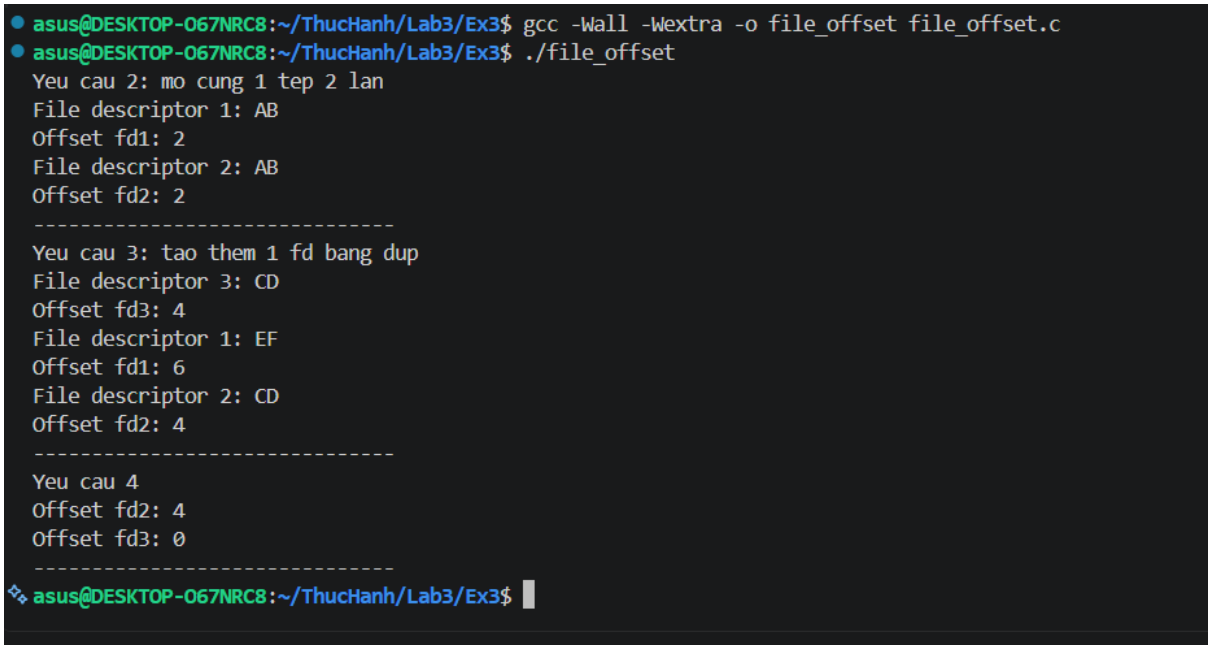


Figure 9: Kết quả chạy chương trình kiểm tra file descriptor và file offset

Thao tác	Dữ liệu đọc	Offset fd1	Offset fd2	Offset fd3
Sau khi mở tệp và tạo fd3	—	0	0	0
Đọc 2 byte từ fd1	AB	2	0	2
Đọc 2 byte từ fd2	AB	2	2	2
Đọc 2 byte từ fd3	CD	4	2	4
Đọc tiếp 2 byte từ fd1	EF	6	2	6
Đọc tiếp 2 byte từ fd2	CD	6	4	6
Đặt offset fd1 về 0	—	0	4	0

Table 3: Dữ liệu đọc và giá trị file offset trong từng thao tác

Nhận xét:

- `fd1` và `fd2` được tạo bởi 2 lời gọi `open()` độc lập có offset riêng do thông tin này được lưu trong `struct file` (chứa cấu trúc thông tin trạng thái mở tệp hiện tại). Khi gọi `open()` độc lập, sẽ tạo độc lập `struct file` riêng và mỗi fd sẽ trỏ đến 1 `struct file` tương ứng.
- Khi sử dụng `dup(fd1)` để tạo `fd3`, bản chất sẽ tạo thêm 1 fd trỏ đến chính `struct file` của `fd1` (cả `fd1` và `fd3` cùng trỏ đến 1 `struct file` chung). Khi 1 fd có thao tác thay đổi thông tin trong `struct file` cũng sẽ ảnh hưởng đến fd còn lại.

- Do đó, thay đổi thông tin qua `fd1` có thể ảnh hưởng đến thao tác tiếp theo trên `fd3`, trong khi `fd2` và `fd1` sử dụng tách biết `struct file` nên không làm thay đổi offset của `fd2`.

3.3 Mối quan hệ giữa các đối tượng

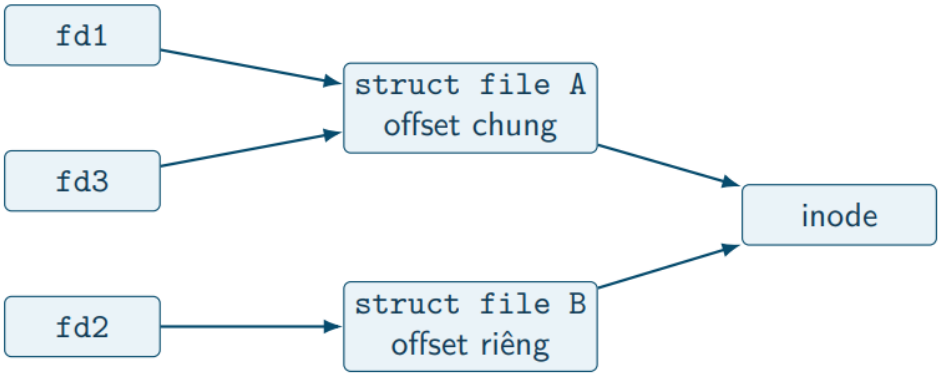


Figure 10: Minh họa mối quan hệ giữa file descriptor, `struct file` và inode

Nhận xét:

- `fd3` được tạo ra bằng `dup(fd1)`; cả 2 fd này đều cùng trỏ đến một `struct file A` duy nhất. Do đó, chúng có chung trạng thái tệp và dùng chung con trỏ vị trí offset.
- `fd2` được tạo độc lập bằng `open()`, do đó nó được cấp phát cho một đối tượng `struct file B` hoàn toàn riêng biệt với `struct file A`. `fd2` duy trì một offset riêng nên không bị ảnh hưởng bởi các thao tác trên `fd1` và `fd3`.
- Dù `struct file A` và `struct file B` đều hoạt động tách biệt nhau, cả 2 đều cùng trỏ về một inode (chứa thông tin trạng thái tệp trong hệ tệp).

4 Bài 4: Phân tích truy cập dữ liệu giữa userspace và kernel

4.1 Phân tích truy cập dữ liệu

1. Vùng nhớ nguồn là `kernel_buf` (thuộc kernel space) và vùng nhớ đích `user buf` (thuộc user space). Dữ liệu được sao chép từ `user_buf` xuống `kernel_buf` (từ user space xuống kernel space).

2. Không nên sử dụng trực tiếp `memcpy()` với `user_buf` do bản chất `memcpy()` chỉ biết copy dữ liệu từ vùng nhớ này đến vùng nhớ khác, chứ không kiểm tra các điều kiện an toàn theo quy tắc của kernel. `user_buf` có thể là một con trỏ không hợp lệ, bị truyền sai, hoặc yêu cầu số byte dữ liệu vượt quá mức tối đa .... Điều này có dẫn đến lỗi hệ thống.

3. Khi `count = 64`, giá trị này vượt qua kích cỡ tối đa mà `kernel_buf` mà thể nhận dữ. Điều này dẫn đến lỗi tràn bộ đệm

4 + 5

```
static ssize_t demo_receive(const char __user *user_buf, size_t count){
    char kernel_buf[32];
    if(count > sizeof(kernel_buf)) return -EINVAL;
    if(copy_from_user(kernel_buf, user_buf, count) != 0) return -EFAULT;
    return count;
}
```

4.2 Truyền dữ liệu từ kernel space sang user space

1. Dữ liệu được truyền từ `kernel_buf` vào `user_buf` (từ kernel space lên user space).

2. Việc sử dụng biến `count` không phù hợp do biến `count` này do người dùng đưa vào, chưa được kiểm tra các điều kiện an toàn và hợp lệ. Biến `count` này có thể nhận một giá trị lớn hơn 5, điều này dẫn đến byte rác ở các byte còn thiếu. Các byte này có thể chứa thông tin quan trọng, gây rò rỉ thông tin.

3. Với trường hợp `count = 20`, lớn hơn độ dài chuỗi ký tự cần sao chép và nhỏ hơn kích thước tối đa của `kernel_buf`, điều này dẫn đến việc có 15 byte rác. Các byte này có thể chứa thông tin quan trọng, gây rò rỉ thông tin.

4 + 5

```
static ssize_t demo_send(char __user *user_buf, size_t count){
    char kernel_buf[32];
    memcpy(kernel_buf, "Linux", 5);
    ssize_t tem = 5;
    ssize_t copy_len = (tem < count) ? tem : count;

    if(copy_to_user(user_buf, kernel_buf, copy_len) != 0)
        return -EFAULT;

    return copy_len;
}
```

4.3 Phân tích kết quả sao chép không hoàn toàn

- Số byte được yêu cầu sao chép là 32.

- Số byte sao chép thành công là 20.
- Số byte chưa được sao chép là 12.
- Có, `-EFAULT` là lỗi không thể truy cập vào vùng nhớ user space theo yêu cầu. Khi thực hiện thao tác `copy_from_user()`, đang copy dở 20 byte thì dừng lại, nguyên nhân do nó chạm đến một vùng địa chỉ không hợp lệ, làm cho quá trình không thể đọc tiếp dữ liệu trong vùng nhớ đó.
- Không thể coi toàn bộ 32 byte trong `kernel_buf` là dữ liệu hợp lệ vừa nhận được vì quá trình copy dữ liệu từ user space xuống kernel space chỉ copy thành công 20 byte, 12 byte còn lại là byte rác, nên dữ liệu trong buffer này cũng trở thành dữ liệu rác, làm sai logic chương trình và không còn giá trị sử dụng nữa.
- Các giá trị trả về không phù hợp do các thao tác thông thường trên user space thường trả về số byte thành công mang giá trị dương. Tuy nhiên, nếu `remaining` lại trả về số byte lỗi chưa được sao chép thì sẽ gây lỗi do hai bên không khớp logic với nhau. Cách xử lý:

```
if(copy_from_user(kernel_buf, user_buf, count) != 0)
    return -EFAULT;
return count;
```

4.4 Tổng hợp kết quả phân tích

Trường hợp	**Vấn đề cần nhận diện**	**Hướng xử lý**
Dùng `memcpy()` với con trỏ userspace	Không kiểm tra các điều kiện an toàn theo quy tắc của kernel. Con trỏ có thể không hợp lệ.	Sử dụng hàm `copy_from_user()` hoặc `copy_to_user()` để thay thế.
`count` vượt quá dung lượng kernel buffer	Vượt qua kích cỡ tối đa mà `kernel_buf` có thể nhận dữ liệu, dẫn đến lỗi tràn bộ đệm (ghi đè làm hỏng dữ liệu stack), gây lỗi hệ thống.	Kiểm tra điều kiện giới hạn trước khi copy: `if(count > sizeof(kernel_buf)) return -EINVAL;`
Sao chép dữ liệu kernel chưa khởi tạo	Sẽ có các byte rác được thêm vào để bù vào các byte còn thiếu. Các byte rác này có thể chứa thông tin quan trọng, gây lỗ hổng rò rỉ thông tin.	Giới hạn biến `count` dựa trên số byte dữ liệu thực tế đã được khởi tạo: `size_t copy_len = (tem < count) ? tem : count;`
Không kiểm tra kết quả của uaccess	Quá trình copy có thể dừng lại giữa chừng (chạm vùng địa chỉ không hợp lệ). Nếu bỏ qua, dữ liệu sẽ bị trộn lẫn byte rác, làm sai logic chương trình và không còn giá trị sử dụng.	Luôn bọc hàm copy trong câu lệnh `if`. Nếu kết quả `!= 0`, trả về lỗi: `return -EFAULT;`
Hiểu sai giá trị trả về của uaccess	Trả về số byte lỗi chưa được sao chép thay vì số byte thành công mang giá trị dương, gây ra lỗi do 2 bên không khớp logic với nhau.	Trả về `-EFAULT` khi có lỗi. Nếu thành công, trả về số byte đã sao chép hợp lệ.

Table 4: Tổng hợp các vấn đề khi truy cập dữ liệu giữa userspace và kernel